INFO212 Systemutvikling · Autumn 2026 · Obligatory assignment 5

Ekte Mat Online on Neo4j

Implement the storage layer of your Oblig 4 application with Neo4j, using Claude Code and GitHub

Group work	Same group as Oblig 4 (2–4 students), one submission per group. Every member commits to the GitHub repository.
Builds on	Obligatory assignment 4: your working Flask application with MVC and repository classes.
Who this is for	Groups whose Oblig 4 report said “Neo4j included: no”. Groups that already used Neo4j in Oblig 4 and said so on their front page do not hand in anything.
Hand in	One short PDF report with a link to your GitHub repository, on Mitt UiB. Deadline: see Mitt UiB.
Assessment	Approved / not yet approved. The assignment is one of the compulsory course requirements.

1 The assignment

In Oblig 4 you built Ekte Mat Online with a temporary data store (SQLite or JSON files) behind repository classes. The finished system is meant to run on Neo4j. In this assignment you make that switch: the application must keep the same functionality, but all data is now stored in and read from a Neo4j graph.

The point is not only to make it run. Ekte Mat’s data is naturally connected: customers place orders, orders contain food items, food items belong to event categories and carry allergens, and customers ask questions about orders. You should model this as a graph and use that in at least some queries.

2 Requirements

Storage layer

- All data from Oblig 4 is stored in Neo4j: food items, image references, event categories, allergens and dietary labels, users (customers and admin), orders with order lines, order and payment status, and questions with answers.
- Only repository classes in the model layer talk to Neo4j. Controllers (blueprints) and views (templates) must not contain Cypher or import the Neo4j driver. If your Oblig 4 design was clean, they should need little or no change.
- Use the official Neo4j Python driver. Use parameterised Cypher (never paste user input into query strings) and transactions for operations that change more than one node or relationship, for example submitting an order with its lines.
- The old SQLite/JSON code is removed from the version you hand in, or clearly separated and unused.

3 Report and submission

Hand in one short PDF (about 2–3 pages) on Mitt UiB that contains:

1. **Front page:** group members, the link to the GitHub repository, the tag of the version to assess (e.g. oblig5), and the Neo4j setup used (Desktop, Docker or Aura). The repository must be public, or private and shared with the course staff.

2. **Graph data model:** a diagram of your nodes, relationships and key properties, and a short explanation of the design choices (for example why allergens are nodes). Say briefly how it differs from your Oblig 4 domain class diagram.